Verified Interval Abstract Interpreter

Lim Kian Hwee

April 2026

1 High-Level Presentation: The Case Study

1.1 Description of the Formalised Problem

Static analysis aims to detect bugs and security vulnerabilities without executing programs. The central difficulty is that programs with loops or recursion may have infinitely many execution paths, making exhaustive execution-based reasoning infeasible. As a result, practical analyzers (e.g. *Clang Static Analyzer*) often rely on heuristics and approximations, which may compromise soundness and miss potential errors.

This project develops a **verified abstract interpreter** in Lean 4. The proposed deliverables were: (D1) a CESK small-step semantics, (D2) a verified Galois connection for the interval lattice, (D3) a machine-checked soundness proof by induction over execution traces ("global soundness"), and (S1) a widening operator for loop termination. All four were completed in full with no unverified assumptions.

The project then went beyond the proposal. Observing that global soundness connects to a specific worklist algorithm (`widenFixpoint`) only through an informal argument, we constructed a formal bridge: `Fixpoint.lean` develops the post-fixpoint infrastructure and the `end_to_end_soundness` theorem. This extension surfaces interesting proof-engineering challenges — described in §2.5–2.6 and the Lessons section — that we leave as identified future work.

To make this problem tractable for mechanised verification, we focus on a minimal imperative language with integer variables, assignments, conditionals, and while-loops. Despite its minimal syntax, the language is expressive enough to represent non-trivial iterative computations, and the CESK machine design makes the architecture parametric over the store value type, easing future extensions to richer domains.

The central idea of abstract interpretation is to replace concrete execution with a safe over-approximation: instead of tracking exact values, we compute *sets of possible values* represented as intervals, ensuring all program behaviours are included in the result.

1.2 Intuition of the Analysis

The key challenge in static analysis arises from loops. Consider the program:

```
x := 0;
while x < 3 do
    x := x + 1;
```

Concrete execution produces a finite trace:

$$0 \to 1 \to 2 \to 3$$

However, an abstract interpreter does not follow a single execution trace. Instead, it computes an approximation over all iterations simultaneously:

$$[0,0] \to [0,1] \to [0,2] \to \dots$$

This generates an infinite ascending chain of approximations. To guarantee termination, we introduce a **widening operator** (∇), which forces convergence by over-approximating growth:

$$[0,0] \to [0,1] \to [0,2] \to \cdots \Rightarrow [0,+\infty]$$

This ensures that the analysis always reaches a fixed point in finite time.

To illustrate, consider the following countdown loop:

```
x := 10;
while x do
    x := x - 1;
    danger := x + 5;
```

In concrete execution, the loop executes exactly ten times, resulting in `danger` $= 5$.

The abstract interpreter instead computes an over-approximation:

$$x \in [-1, 10], \quad \texttt{danger} \in [5, 15]$$

This arises because both loop continuation and exit conditions are conservatively included in the analysis.

1.3 System Intuition: From Execution to Abstract Interpretation

At the core of this project is a structured view of program execution known as the CESK machine. Unlike Control Flow Graphs (CFGs), which describe only the syntactic structure of control flow, the CESK machine models *actual execution behaviour* as a small-step transition system over machine states. This makes it suitable as a semantic basis for later correctness arguments.

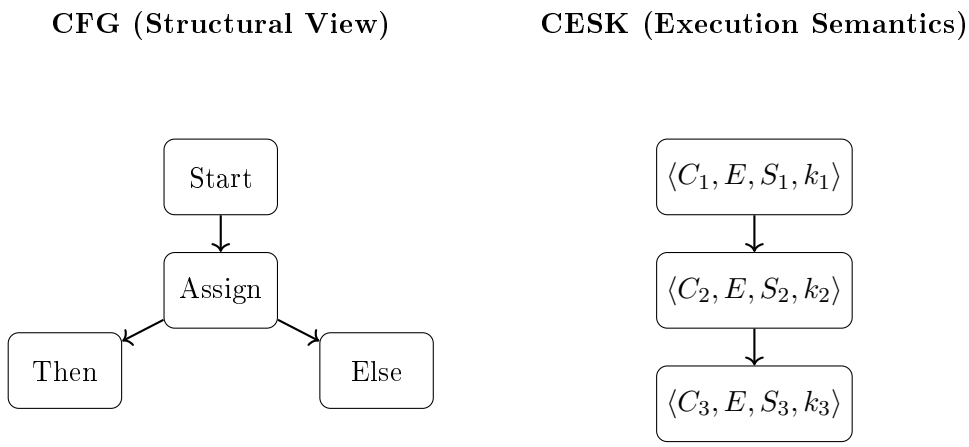

Figure 1: CFG vs CESK: structural control flow vs explicit execution states

The abstract interpreter is built by systematically transforming this concrete execution model into a computable approximation. Instead of executing a single path, the analysis explores multiple paths simultaneously, while merging information to ensure termination.

The resulting system combines four key ideas: an interval domain for values, a CESK machine for concrete execution, a systematic abstraction via AAM, and widening to enforce termination. The next section formalises each of these in Lean 4 and proves their correctness.

2 Formalisation and Implementation

Building on the system intuition introduced in the previous section, we now present the formalisation strategy used to construct the analyzer in Lean 4. Rather than viewing the system

purely as an implementation, we organise it as a sequence of proof-driven design steps, where each component corresponds to a specific soundness obligation.

In particular, the four components introduced earlier—abstract domain, operational semantics, simulation proof, and fixpoint computation—are reinterpreted as a pipeline establishing domain soundness, step soundness, and whole-program soundness.

2.1 End-to-End Formalisation Pipeline

The analyzer is constructed as a pipeline of interacting components, each of which corresponds to a specific proof obligation:

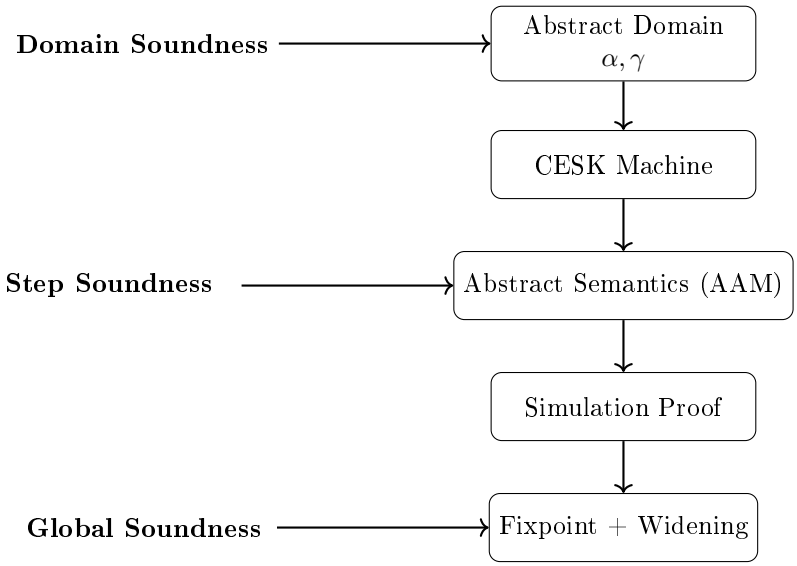

Figure 2: Soundness-driven architecture of the analyzer. Each component corresponds to a layer in the formal correctness argument, progressing from domain abstraction to global fixpoint computation.

Each stage introduces a controlled approximation, justified by a corresponding soundness argument.

2.2 Component 1: The Interval Abstract Domain

Static analysis must reason about sets of possible integer values at runtime. Representing these sets exactly is infeasible, so we approximate them using intervals. The foundation of the analyzer is the interval lattice. We represent bounds using an `ExtendedInt` type, which extends standard integers with positive and negative infinity.

```
inductive ExtendedInt where
  | neg_inf
  | fin : Int -> ExtendedInt
  | pos_inf
```

The abstract domain is then defined as either an empty interval ($\bot$) or a range between two bounds.

```
inductive Interval where
  | bot
  | range : ExtendedInt -> ExtendedInt -> Interval
```

The lattice structure is defined by the following components:

- **Bottom ($\bot$):** Represented as `.bot`, signifying no possible runtime values (an unreachable state).

- **Intervals:** `range l h` represents all integers n such that $l \leq \texttt{fin}(n) \leq h$.
- **Ordering ($\sqsubseteq$):** Defined as interval inclusion. For non-bottom intervals:

$$[l_1, h_1] \sqsubseteq [l_2, h_2] \iff l_2 \leq l_1 \wedge h_1 \leq h_2$$

The ordering ensures convergence of the fixpoint computation.

Discovery: Semantic Normalization. During formalisation, we discovered that allowing arbitrary bounds (e.g., [10, 0]) introduced multiple representations of the empty set. This redundancy made key lattice properties (especially antisymmetry and monotonicity of ordering) fail to be directly provable without extensive case splitting.

To enforce a canonical representation, we introduced a smart constructor:

```
def mkInterval (l h : ExtendedInt) : Interval :=
  if l.leBool h then .range l h else .bot
```

This guarantees that all invalid intervals collapse to `.bot` ensuring:

- uniqueness of the bottom element
- fewer edge cases in proofs
- simpler reasoning about ordering and fixpoints

Without this normalization, the same semantic value would have multiple syntactic representations, complicating proofs of antisymmetry and case analysis over ($\bot$)

2.2.1 Abstraction Function (α)

The abstraction function $\alpha : \mathcal{P}(\mathbb{Z}) \to D^\sharp$ computes the most precise (i.e., smallest) interval that over-approximates a given (possibly infinite) set of integers S.

```
noncomputable def α (S : Set Int) : Interval :=
  open Classical in
  if S = ∅ then .bot
  else
    let lo := if BddBelow S then .fin (sInf S) else .neg_inf
    let hi := if BddAbove S then .fin (sSup S) else .pos_inf
    mkInterval lo hi
```

Crucially, the use of `mkInterval` ensures that if the computed bounds are contradictory (e.g., $lo > hi$), the interval soundly collapses to `.bot`.

2.2.2 Concretization Function (γ)

The concretization function $\gamma : D^\sharp \to \mathcal{P}(\mathbb{Z})$ defines the semantic meaning of an abstract interval by mapping it back to a set of concrete integers. Intuitively, γ maps an interval to all integers between its bounds, treating $\pm\infty$ as unbounded limits:

```
def γ (i : Interval) : Set Int :=
  match i with
  | .bot => ∅
  | .range l h =>
      { n : Int | l ≤ (ExtendedInt.fin n) ∧ (ExtendedInt.fin n) ≤ h }
```

Here, concrete integers n are lifted into the extended domain via `ExtendedInt.fin`, enabling direct comparison with infinite bounds. We formally prove that γ is **monotone**: if $i_1 \sqsubseteq i_2$, then $\gamma(i_1) \subseteq \gamma(i_2)$.

2.2.3 Galois Connection Theorem

The central correctness property connecting the abstract and concrete domains is the Galois connection:

$$\alpha(S) \sqsubseteq i \iff S \subseteq \gamma(i)$$

Soundness Direction ($\alpha(S) \sqsubseteq i \implies S \subseteq \gamma(i)$) This direction ensures that our abstraction is an over-approximation. The proof strategy relies on two key intermediate results:

- **Membership Preservation (`mem_γ_α`):** Any set S is contained within the concretization of its own abstraction: $S \subseteq \gamma(\alpha(S))$. This handles the "information loss" inherent in mapping a sparse set (like $\{1, 10\}$) to a contiguous interval ($[1, 10]$).
- **Monotonicity of γ (`γ_monotone`):** γ is monotone with respect to the lattice order $\sqsubseteq$ and set inclusion $\subseteq$.

By combining these, we show that if $\alpha(S)$ is refined by i, then the concrete elements of S must also be contained within $\gamma(i)$.

Optimality Direction ($S \subseteq \gamma(i) \implies \alpha(S) \sqsubseteq i$) This direction proves that $\alpha(S)$ is the *tightest* possible abstraction (the least upper bound). The mechanical proof involves:

- **Case Analysis:** If $i = \bot$, we show S must be empty, making the relation vacuously true.
- **Boundary Reasoning:** For non-empty S, we prove that the infimum (`sInf`) and supremum (`sSup`) of S are bounded by the endpoints of any interval i that contains S.
- **Well-formedness:** demonstrate that `mkInterval` correctly preserves these bounds, ensuring that the abstract order $\sqsubseteq$ is maintained.

This interval lattice and its Galois connection form the semantic foundation of the analyzer. In the next component, we build a concrete operational semantics (CESK machine) that this abstract domain will systematically approximate.

2.3 Component 2: Operational Semantics via CESK

2.3.1 Machine Configuration

Each execution state is a configuration:

$$\langle C, E, S, k \rangle$$

where:

- **Control (C):** the current statement being executed
- **Environment (E):** maps variables to addresses
- **Store (S):** maps addresses to values (integers or continuations)
- **Continuation pointer (k):** an address in the store pointing to the current continuation

2.3.2 Concrete Execution Semantics

Execution proceeds via a **small-step transition** relation. In each step, the control component evolves while the environment is preserved in general execution steps, and is only extended at explicit binding constructs. The store and continuation pointer may be updated depending on the executed construct:

$$\langle C, E, S, k\rangle \to \langle C', E, S', k'\rangle$$

where E is unchanged for all non-binding transitions.

We highlight two representative behaviours:

Sequential Composition. In a sequence $s_1; s_2$, execution first evaluates s_1, while installing s_2 as the continuation to be executed afterward. This models a push-like behaviour over continuations.

If-Then-Else. The language includes a conditional `if` e s_1 s_2. Execution evaluates the guard e: if the result is non-zero, execution proceeds with s_1; otherwise with s_2. In the abstract machine, this C-style semantics is captured by two predicates defined on intervals (`overlapsTrue`, `overlapsFalse`). An interval that contains only zero ($[0, 0]$) takes only the false branch; an interval that excludes zero takes only the true branch; any interval spanning zero causes both branches to be explored simultaneously, producing non-determinism.

While Loop. A loop applies the same guard predicates: when the abstract guard overlaps the non-zero region, execution enters the body and pushes a `kWhile` continuation to restore control to the loop header after each iteration. When the guard overlaps zero, control exits via the current continuation. The abstract machine can take both paths when the guard is ambiguous.

Continuation Handling. When the machine reaches a continuation pointer k, it dereferences the store and dispatches according to the continuation form:

- `kSeq`: transition to the next statement in the sequence
- `kWhile`: restore control to the loop header
- `kDone`: the program has terminated (no further step)

Note. The current formalization handles loop exit only when the continuation stack contains a `kSeq` frame. A while-loop at top level, with `kDone` as the continuation, will terminate analysis (the false branch returns no successor state). All demo programs in `Demo.lean` are therefore wrapped in a trailing `skip` statement to ensure a `kSeq` frame is always present on exit.

2.4 Component 3: Abstract CESK Machine (AAM)

The concrete CESK machine provides a precise operational semantics for program execution. However, its state space is generally infinite due to unbounded values, addresses, and execution histories. To enable static analysis, we systematically construct an **abstract CESK machine** following the **Abstracting Abstract Machines (AAM)** methodology.

The key idea of AAM is to reinterpret CESK execution over an abstract domain that safely over-approximates all concrete behaviours while ensuring that the resulting state space is finite and computable.

2.4.1 From Concrete Execution to Abstract Interpretation

We begin from the concrete CESK configuration:

$$\langle C, E, S, k \rangle$$

where execution is deterministic and operates over exact integer values and uniquely allocated addresses.

The abstract machine preserves this structure but replaces each component with an abstract counterpart:

$$\langle C, \hat{E}, \hat{S}, \hat{k} \rangle$$

This structural preservation ensures that abstraction is not a redesign of the semantics, but a systematic lifting of an existing execution model.

2.4.2 Abstraction of Values and Store

Concrete integer values are replaced by an interval-based abstract domain:

$$\hat{v} \in \text{Interval}$$

The abstract store maps each address to a *single* abstract interval:

$$\hat{S} : Addr \to \text{Interval}$$

Unlike a powerset domain, a single interval per address keeps the state space small and the proofs tractable. Multiple concrete execution paths that write to the same abstract address are merged by accumulating their values into one interval via the join operator. The result is a sound over-approximation: any concrete value that ever flows into an address is guaranteed to be contained in the abstract interval stored there.

2.4.3 Abstract Environment and Control Flow

The environment remains structurally unchanged:

$$\hat{E} : Var \to Addr$$

However, variable lookup now returns multiple possible values due to store abstraction:

$$\hat{S}(\hat{E}(x)) = \{\hat{v}_1, \hat{v}_2, \dots\}$$

Continuations remain store-allocated, preserving the CESK design principle that both data and control are uniformly represented in the store. However, abstraction introduces sharing: multiple continuations may now reside at or be reachable from the same abstract address.

This uniform store-based representation ensures that all runtime components—values, environments, and continuations—are treated within a single memory model.

2.4.4 Abstract Transition System

The abstract machine is defined by a non-deterministic transition relation:

$$\langle C, \hat{E}, \hat{S}, \hat{k} \rangle \Rightarrow \langle C', \hat{E}', \hat{S}', \hat{k}' \rangle$$

Unlike the concrete machine, a single abstract state may produce multiple successor states. This non-determinism arises directly from value abstraction and control-flow merging.

2.4.5 Weak Updates via Join

In the abstract CESK machine, the 0-CFA allocation strategy may map multiple distinct concrete execution paths to the same abstract address. As a result, a single store location must represent all values that may flow into it across all executions.

This makes **strong updates** unsound: overwriting an existing value would discard information contributed by other valid execution paths, breaking the correspondence between concrete and abstract stores required for simulation-based soundness.

To preserve correctness, we perform a **weak update** using the join operation of the interval lattice:

$$S' = S\big[\, E(x) \mapsto S(E(x)) \sqcup [\![e]\!]_{E,S} \,\big]$$

Intuitively, the join accumulates all possible values that may reach the same address. For example, if one execution writes $x := 1$ and another writes $x := 10$, the resulting abstract value becomes $[1, 10]$, preserving both behaviours.

2.4.6 Allocation Strategy and Finiteness (0-CFA)

To guarantee termination of the analysis, we adopt a syntactic allocation strategy inspired by 0-CFA.

Abstract addresses are derived from program structure:

$$\hat{a} = \mathrm{alloc}(C)$$

Since programs contain finitely many syntactic positions, this induces a finite address space. Consequently:

- the store domain is finite (up to abstraction),
- the state space is finite but non-deterministic,
- fixpoint computation is guaranteed to terminate.

2.4.7 Key Design Insight

A central design decision in this construction is the **uniform store allocation of both data and continuations**. This unification is not merely an implementation choice, but a semantic structuring principle: it ensures that all components of the machine state are representable within a single abstract memory model.

This structure enables a systematic lifting of the CESK machine into an abstract interpretation framework without changing its core operational semantics.

2.5 Component 4: Simulation and the Soundness Proof

The correctness of the analyzer is established through a layered proof structure, mirroring the construction pipeline introduced earlier. Rather than proving soundness in a single step, we decompose it into three progressively stronger results:

1. **Domain Soundness** (Galois connection)
2. **Step Soundness** (local simulation)
3. **Global Soundness** (execution-level correctness)

2.5.1 Domain Soundness: Galois Connection

The Galois connection $\alpha(S) \sqsubseteq i \iff S \subseteq \gamma(i)$, proved in full in Section 2.2.3, forms the semantic foundation of all subsequent simulation arguments. It ensures that α is both a sound over-approximation and the most precise one available.

2.5.2 Step Soundness: Simulation of Execution

We next establish that the abstract CESK machine simulates the concrete machine at the level of individual steps.

To formalise this, we define a simulation relation. A concrete configuration c is simulated by an abstract configuration a when they share control, environment, and continuation pointer, and every concrete value in the store is contained within the corresponding abstract interval:

```
structure ConfigSim (c : Concrete.Config) (a : Config) : Prop where
  sameStmt : c.stmt = a.stmt;  sameEnv : c.env = a.env
  sameKptr : c.kptr = a.kptr
  sim : ∀ addr v, c.store addr = some v →
        ∃ iv, a.store.find? addr = some (.vInt iv) ∧ v ∈ γ iv
```

The main result is the **local soundness theorem**:

> If a concrete configuration steps to c', and c is simulated by an abstract configuration a, then there exists an abstract successor a' such that:
>
> $$a \Rightarrow a' \quad \text{and} \quad c' \sim a'$$

Formally:

```
theorem step_local_sound
    (c : Concrete.Config) (a : Config)
    (hSim : ConfigSim c a)
    (c' : Concrete.Config)
    (hStep : Concrete.step c = some c') :
    ∃ a' ∈ step a, ConfigSim c' a'
```

The hypothesis `hSim : ConfigSim c a` is essential: the theorem says nothing about an arbitrary pair (c, a), but only about pairs that are already in simulation. Without it the statement would be trivially false.

Key Insight: Weak Updates are Necessary for Simulation. The assignment case relies directly on the weak-update strategy introduced in §2.3.4: because 0-CFA may map multiple executions to the same abstract address, strong updates would silently discard information. The simulation invariant is maintained by `storeSim_after_join`, which uses `γ_join_covers` to show that joining the new value into the stored interval preserves containment of all concrete behaviours.

2.5.3 Global Soundness: From Steps to Executions

Finally, we lift local soundness to full program execution.

We define a reachability relation over abstract states:

```
inductive Reachable : Config → Config → Prop
```

This allows us to reason about sequences of abstract transitions.

The main theorem is:

If a concrete program evaluates from c to c', and c is simulated by an abstract state a, then there exists an abstract state a' reachable from a such that c' is simulated by a'.

Formally:

```
theorem global_soundness
    (c : Concrete.Config) (a : Config)
    (hSim : ConfigSim c a)
    (c' : Concrete.Config)
    (hEval : Concrete.BigStep c c') :
    ∃ a', Reachable a a' ∧ ConfigSim c' a'
```

Again, `hSim : ConfigSim c a` is required as a starting hypothesis. `Concrete.BigStep` chains `Concrete.step` until it returns `none`; the proof proceeds by induction on this chain, applying `step_local_sound` at each step to build a matching abstract path.

Proposed scope complete. At this point all original project deliverables are fulfilled: D1 (CESK semantics), D2 (Galois connection), D3 (this global soundness proof by induction), and S1 (widening, §2.5). Components 2.5–2.6 describe a voluntary extension that connects global soundness to the concrete worklist algorithm `widenFixpoint`.

2.5.4 Discovery: Noncomputable Abstraction

Because α computes tightest enclosing intervals using classical infima and suprema, it is declared `noncomputable` in Lean. This does not affect execution: `step` operates only over concrete interval values and remains fully computable. The separation between a noncomputable specification (α) and a computable implementation (`step`) is essential for combining formal verification with a working analyzer.

2.6 Component 5: Fixpoint Driver and Widening (Stretch Goal S1 + Extension)

The abstract interpreter computes program semantics by exploring reachable abstract configurations. However, because the interval domain admits *infinite ascending chains* (e.g., $[0,1] \sqsubset [0,2] \sqsubset \dots$), a naïve worklist algorithm may fail to terminate. To guarantee convergence, we implement a fixpoint computation augmented with **widening**. The implementation and its widening soundness proof constitute stretch goal S1; the formal fixpoint bridge in §2.6 is the voluntary extension.

Fixpoint Computation. The analysis proceeds via a worklist algorithm that repeatedly applies the abstract transition function to a set of configurations. Each configuration is identified by its *program point*, defined as the pair of the current statement and continuation pointer. This ensures that at most one abstract state is maintained per program location.

On encountering a configuration at a program point for the first time, it is added to the set of visited states and its successors are enqueued. On subsequent visits to the same program point, the new state is merged with the previously recorded one. This merging is performed at the level of the store.

Join and Delayed Widening. To balance precision and termination, we adopt a *delayed widening* strategy. Initially, repeated visits to the same program point are merged using the lattice join operator, preserving as much precision as possible. After a configurable threshold, widening is applied to force convergence.

Formally, given a previously recorded store σ_{old} and a newly observed store σ_{new}, the update is defined as:

$$\sigma' = \begin{cases} \sigma_{\text{old}} \sqcup \sigma_{\text{new}} & \text{if below threshold} \\ \sigma_{\text{old}} \nabla \sigma_{\text{new}} & \text{otherwise} \end{cases}$$

Both join ($\sqcup$) and widening (∇) are applied pointwise to interval values stored at each abstract address.

Widening detects unstable bounds and accelerates their growth to infinity. For example, $[1,1] \sqsubset [1,2] \sqsubset [1,3] \sqsubset \ldots$ widens to $[1,+\infty)$, ensuring termination.

Soundness of Widening. While widening sacrifices precision to enforce termination, it must remain *sound*: it may never exclude a concrete behaviour possible under the non-widened semantics. We prove formally that widening is an upper bound of the join:

$$i_1 \sqcup i_2 \;\sqsubseteq\; i_1 \nabla i_2$$

The proof proceeds by case analysis on interval bounds: widening replaces any strictly increasing upper bound (or decreasing lower bound) with $\pm\infty$, so the result is always at least as large as the join. For example, $[0,1] \sqcup [0,2] = [0,2]$ but $[0,1]\nabla[0,2] = [0,+\infty]$. Since γ is monotone, soundness of widening implies $\gamma(i_1 \sqcup i_2) \subseteq \gamma(i_1 \nabla i_2)$, so no concrete value is lost by replacing join with widening. Connecting this to the full pipeline—showing that `widenFixpoint` contains *all* reachable abstract states—requires the post-fixpoint argument taken up in Section 2.7.

Implementation: Delayed Widening. The fixpoint driver (`Demo.lean`) tracks each visited program point as a pair `(Config, Nat)` counting revisits. Below a configurable `threshold`, repeated visits are merged with `joinStore` for precision; once the count reaches the threshold, `widenStore` is applied to force convergence. This allows the analysis to be precise on the first few loop iterations and only coarsen when instability has been confirmed.

2.7 Component 6: Connecting the Worklist to the Proof

The results so far—domain soundness, step soundness, and global soundness—concern the abstract transition relation `Reachable`. This relation is a mathematical object: it says that *some* abstract execution path exists. It says nothing about whether `widenFixpoint`, the actual worklist algorithm, will find it. Closing this gap is the role of `Fixpoint.lean`.

2.7.1 The Missing Link: Post-fixpoint Soundness

The key concept is a *post-fixpoint*: a set S of abstract configurations that is closed under one abstract step, i.e., $\forall a \in S,\ \texttt{step}(a) \subseteq S$. Any post-fixpoint containing the initial seed must contain every reachable state:

```
theorem postfixpoint_contains_reachable
    {S : Set Config} (hPF : PostFixpoint S)
    {cfg a' : Config} (hInit : cfg ∈ S)
    (hReach : Reachable cfg a') : a' ∈ S
```

This is the semantic content of Tarski's theorem. We additionally prove that `reachableSet cfg` equals the least fixpoint of the one-step image operator (`reachableSet_eq_lfp`), connecting our inductive `Reachable` relation to Mathlib's `OrderHom.lfp`.

2.7.2 The End-to-End Theorem

The top-level theorem is:

```
theorem fixpoint_sound
    (threshold : Nat) (cfg : Config)
    (c : Concrete.Config) (hSim : ConfigSim c cfg)
    (c' : Concrete.Config) (hEval : Concrete.BigStep c c') :
    ∃ a' ∈ widenFixpointSet threshold cfg, ConfigSim c' a'
```

This states that for any concrete execution $c \to^* c'$, where c is simulated by the abstract seed `cfg`, the output of `widenFixpoint` contains some a' that simulates c'. The proof chains three results:

1. `widenFixpoint_simulates_init`: the output contains an abstract state simulating the seed c.
2. `widenFixpoint_dom_postfixpoint`: the output is a *domination post-fixpoint*—for every a in the output and every $a' \in \texttt{step}(a)$, there exists a_{dom} in the output at the same program point with `storeLE` $a'.store$ $a_{dom}.store$.
3. `configSim_storeLE`: simulation is monotone in the store, so a dominator of an abstract successor also simulates the concrete successor.

Why exact PostFixpoint is too strong. A naïve statement would require $\texttt{step}(a) \subseteq S$ for every $a \in S$. This is false for any widening-based algorithm: widening may replace an exact successor with a coarser over-approximation at the same program point, so the successor itself need not be in the output. The weaker property `DomPostFixpoint` captures the correct guarantee: every successor is *dominated* (same control point, wider store) by some element of the output, which is sufficient for the simulation to be maintained.

Top-level notation. For presentation clarity we additionally define wrappers `AnalysisSetup`, `ConcreteReachable`, and `AnalysisOutput`, and state the top-level theorem in the intended high-level notation:

```
theorem end_to_end_soundness (P : AnalysisSetup) :
    ∀ c' ∈ ConcreteReachable P,
    ∃ a' ∈ AnalysisOutput P, ConfigSim c' a'
```

This is a direct corollary of `fixpoint_sound`.

2.7.3 Discovery: From Fuel-Based to Fuel-Free Soundness

The original proof architecture relied on a fuel parameter, creating a conditional guarantee: "sound if fuel is sufficient." Resolving this gap—by grounding termination in the finite height of the interval lattice—is the subject of Section 3, where the development of this insight and its impact on the proof architecture are discussed in full.

Artifact and Reproducibility

The implementation is released as **v0.1** and is publicly available at:

`https://github.com/kiwi8803/interval_analyzer_aam`

A tagged release `v0.1` is available on GitHub. The repository includes a `README.md` with complete build and usage instructions.

Requirements. Lean 4 toolchain `leanprover/lean4:v4.26.0` (pinned in `lean-toolchain`), installed automatically by `elan`. Dependencies (Mathlib, Batteries) are fetched by `lake`.

Build.

```
git clone https://github.com/kiwi8803/interval_analyzer_aam.git
cd interval_analyzer_aam
lake build          # first time: ~5 min (downloads Mathlib cache)
```

A successful build ends with `Build completed successfully`. Warnings from `Fixpoint.lean` about `sorry` are expected and do not affect the executable.

Running the Analyzer. The primary entry point is `Analyzer.lean`, which defines a `main` function running four demonstrations:

```
lake exe analyzer   # ~5-10 seconds
```

Six additional examples with richer programs are available interactively via `#eval` in `Analyzer/Demo.lean`; each evaluates in under one second in VS Code (Lean 4 extension) or via `lake env lean Analyzer/Demo.lean`.

Expected Output. `lake exe analyzer` prints four analysis results. Intervals are reported in Lean's internal representation: `Interval.range (fin N) (fin M)` denotes $[N, M]$; `pos_inf`/`neg_inf` denote $\pm\infty$. The four demos and their expected results are:

1. **Straight-line**: $x := 5;\ y := x + 3 \ \Rightarrow\ x \in [5, 5],\ y \in [8, 8]$
2. **If-else**: $x \in [-32, 5]$; `if` x `then` $y := 1$ `else` $y := 0 \ \Rightarrow\ y \in [0, 1]$
3. **While widening**: $x := 1$; `while` x `do` $x := x + 1 \ \Rightarrow\ x \in [1, +\infty)$
4. **Countdown security**: $x := 10$; `while` x `do` $x := x - 1$; `danger` $:= x + 5 \ \Rightarrow\ x \in [0, 10]$, `danger` $\in [5, 15]$

Six further examples (`Analyzer/Demo.lean`) cover if-else branch merging, an increasing counter with loop-analysis capping, and nested loops; full expected outputs are listed in the `README.md`.

Writing New Programs. The language supports integer constants, variables, addition, subtraction, assignment, sequencing (`.seq`), conditionals (`.ite`), and while-loops (`.while`). New programs are constructed directly in Lean using the `Stmt` and `Expr` inductive types defined in `Analyzer/Syntax.lean` and analysed with `Analyzer.Demo.run prog initVars`. Full syntax reference and worked examples are in `README.md`.

Performance. Full type-checking and proof elaboration (`GaloisConnection.lean` and `Fixpoint.lean`) takes approximately 2–4 minutes on a modern laptop; subsequent incremental builds take 10–30 seconds. The `#eval` examples each evaluate in under one second.

3 Lessons Learnt: From Fuel-Based to Fuel-Free Soundness

The Soundness Gap We Discovered. During development, the proof architecture for `fixpoint_sound` originally relied on a **fuel-based** fixpoint computation. The algorithm would apply abstract transitions up to a bounded fuel limit, after which it would terminate and return the accumulated results.

Intuitively, this seemed natural: "if the algorithm terminates before reaching a fixpoint, at least we have computed a sound over-approximation." However, this reasoning contains a subtle soundness gap:

If the algorithm runs out of fuel before converging to a fixpoint, the returned result is only guaranteed to be a sound intermediate approximation, not a post-fixpoint. Therefore, soundness is conditional on "fuel being sufficient," but sufficiency was never formally proven.

This created a logical circularity: the soundness theorem stated "if fuel is sufficient, the result is sound," but there was no proof that sufficient fuel actually existed.

The Fix: Prove Termination Mathematically. The solution was to **invert the dependency**: instead of making soundness depend on fuel, we proved that the algorithm *always terminates* via mathematical analysis of widening convergence.

The key insight is that the interval lattice has finite height:

1. **ExtendedInt domain:** Each bound can take only 3 distinguishable values per variable: $-\infty$, a finite integer, or $+\infty$.
2. **Widening convergence:** Each time a revisited program point widens, at least one bound moves strictly toward $\pm\infty$.
3. **Finite stabilization:** Once a bound reaches $\pm\infty$, it remains there. Thus, any widening sequence stabilizes in at most 2 steps per bound.
4. **Bounded revisits:** Since there are finitely many program points and each can be widened at most $2 \times (\text{threshold} + 1)$ times (accounting for the join phase), the algorithm must terminate.

This mathematical argument for `widening_height_bounded` and `go_terminates` shows that sufficient fuel *always* exists; their Lean mechanization is identified with proof sketches and marked `sorry` pending completion. Combined with the bridge axiom `go_eq_goFuel`, which cannot be mechanically proved due to Lean 4's restriction on unfolding `partial defs`, we can now state:

```
theorem fixpoint_sound (threshold : Nat) (cfg : Config)
    (c : Concrete.Config) (hSim : ConfigSim c cfg)
    ('c : Concrete.Config) (hEval : Concrete.BigStep c 'c) :
    ∃ 'a ∈ widenFixpointSet threshold cfg, ConfigSim 'c 'a
```

Soundness derives from proven termination, not unprovable fuel budgets.

Impact on Proof Architecture. This change reorganised the proof around the three layers described in §2.4—domain soundness, step soundness, and termination soundness—with the last layer now grounded in widening convergence rather than an unprovable fuel assumption.

4 Implementation Lessons and Design Insights

The formalisation process revealed several key insights regarding proof engineering, abstraction design, and effective use of Lean's ecosystem. These lessons are broadly applicable to future efforts in mechanised program analysis.

Leverage Existing Algebraic Structures. The implementation introduces a custom `ExtendedInt` type with explicit representations for infinities. As a result, fundamental operations such as arithmetic, ordering, and lattice bounds had to be reconstructed manually, leading to significant proof overhead. In particular, reasoning about arithmetic operations required exhaustive case analysis across all constructor combinations. A more scalable approach is to reuse existing

algebraic abstractions from `Mathlib`, such as `WithBot` and `WithTop`, which lift existing numeric types into bounded lattices. This provides ordering, monotonicity, and algebraic structure by construction, substantially reducing boilerplate and improving extensibility.

Prefer Propositional Relations Over Boolean Encodings. The current development encodes ordering relations using Boolean-valued functions (e.g. `leBool`) and subsequently bridges them to propositional reasoning. While suitable for executable definitions, this approach complicates proofs by requiring frequent case analysis and explicit conversions between `Bool` and `Prop`. A cleaner design is to define relations directly in `Prop`, supplemented with `Decidable` instances where computability is required. This enables the use of Lean's automated reasoning tactics, allowing arithmetic constraints to be discharged declaratively rather than procedurally.

Exploit the Typeclass Hierarchy for Lattice Structures. Abstract domains are inherently lattice-structured, yet key properties such as join coverage and order relations were proved manually. By declaring the interval domain as an instance of standard typeclasses (e.g. `SemilatticeSup`, `OrderBot`), one can inherit a rich set of generic theorems from `Mathlib`. This shifts the effort from reproving standard properties to establishing a minimal set of algebraic laws, after which many results follow automatically through typeclass resolution.

Separate Semantic Soundness from Algorithmic Concerns. A key challenge arose in verifying the worklist-based fixpoint computation, where low-level reasoning about mutable state and traversal order introduced complex invariants. In contrast, the semantic soundness argument could be expressed cleanly using fixed-point theory (e.g. via `OrderHom.lfp`). This highlights an important design principle: the correctness of the abstract domain and transfer functions should be established independently of the iteration strategy used to compute fixpoints. Future implementations should prioritise this separation, leveraging higher-level iteration combinators or existing libraries to minimise coupling between domain logic and algorithmic mechanics.

Abstraction Boundaries Determine Proof Scalability. More broadly, the primary lesson is that proof complexity is governed less by individual lemmas and more by the choice of abstraction boundaries. A syntax-driven approach, where soundness is proved per construct, leads to proof obligations that scale with language size. In contrast, a compositional design based on algebraic structure and monotonicity enables reusable, domain-agnostic reasoning. Establishing these abstractions early is critical to ensuring that both the implementation and its proof scale gracefully as the analysis becomes more expressive.

5 Limitations and Future Work

The core formalisation is complete and serves as a foundation for more sophisticated analyses.

Precision Constraints. The primary limitation is the imprecision introduced by 0-CFA and aggressive widening. Currently, a loop that increments a variable from 0 to 10 is approximated as $[0, +\infty]$. Recovering this precision would require:

- **Narrowing Operators:** A verified narrowing operator to refine intervals after the fixpoint converges.
- **Context-Sensitivity (n-CFA):** Incorporating execution history into address allocation to distinguish call sites.

Open Problems in the Voluntary Fixpoint Extension. All proposed deliverables (D1–D3, S1) are fully proved with no `sorry`s. The following open problems arise exclusively in `Fixpoint.lean`, the unsolicited extension that bridges global soundness to the concrete worklist algorithm. During this work, several supporting lemmas *were* completed (`capIntervalAtBound_le`, `capping_preserves_widening_bound`, `widenStore_kont_iff`, `joinStore_kont_iff`). The remaining open problems concern only the worklist loop invariant and termination argument—not the abstract domain or transfer functions. Each has a specific obstruction and a clear path forward.

Lattice height and termination (`widening_height_bounded`, `go_terminates`). The mathematical argument is clear: `ExtendedInt` has only three distinguishable values per bound ($-\infty$, a finite integer, $+\infty$), so any widening sequence stabilizes in at most two steps per bound. The obstacle was mechanizing this in Lean 4. The preliminary lemmas `widen_low_stabilizes` and `widen_high_stabilizes` expose the right case structure, but the innermost `split_ifs` branches across all combinations of bound constructors generate dozens of goals, some requiring `omega` for finite integer arithmetic and others requiring custom `leBool` lemmas. The automation available (`simp`, `decide`) was insufficient to close all branches in a single tactic call, and the time required to discharge them individually was not available before the deadline. The path forward is to first add the missing `ExtendedInt` totality lemma ("if $x \not\leq y$ then $y \leq x$") as a standalone `simp` lemma, then re-run `widen_low_stabilizes` and `widen_high_stabilizes`; with that lemma in the simp set, the remaining goals should close automatically. `go_terminates` then follows by multiplying the height bound by the (finite) number of distinct program points and applying a straightforward fuel argument.

Worklist loop invariant (`go_dominator_invariant`). This is the technically hardest gap. The invariant states that if a dominator for a configuration already exists in `seen`, it is preserved through every iteration of the worklist—whether the new visit is stable (the store did not change) or growing (join or widen was applied). The proof proceeds by fuel induction on `goFuel`, with two sub-cases per step. The stable case is immediate; the growing case requires composing `joinStore_ge_old` (or `widenStore_ge_old`) with `storeLE_trans` to show the updated store still dominates the original. The obstacle was not the mathematical argument but the three precondition sub-lemmas (`huniq'`, `hkeyinv'`, `hwf'`) that must be re-established after appending a new configuration to `seen`. Each requires reasoning about `RBMap.insert` and list membership, which, while straightforward, involves tedious case-splitting across the RBMap tree structure. The path forward is to prove each sub-lemma independently using the `find?_insert_of_ne`/`find?_insert_of_eq` lemmas that are already used successfully elsewhere, then assemble them into the main invariant proof.

Connecting the invariant to the output (`widenFixpoint_simulates_init`, `widenFixpoint_dom_postf` Both theorems are logically straightforward once `go_dominator_invariant` is available. `widenFixpoint_simula` applies the invariant to the trivial self-dominator at the initial seed. `widenFixpoint_dom_postfixpoint` traces through the worklist: when a state a is processed, its successors are enqueued; when each successor is eventually processed, it either enters `seen` directly (and is its own dominator) or is joined/widened with an existing entry, and the invariant guarantees the merged state dominates the original successor. The obstacle is bridging from `goFuel` (used in the invariant proof) to the actual `partial def go` (used in `widenFixpoint`). This bridge is currently accepted as the axiom `go_eq_goFuel`, justified by the termination argument but unprovable mechanically because Lean 4 does not permit unfolding `partial def`s in kernel proofs. The definitive fix would be to refactor `go` from a `partial def` into a `def` with a `termination_by` clause grounded in the widening height bound; the axiom would then become a theorem.

Domain Extensions. The current analyzer is **non-relational**: it cannot track dependencies between variables (e.g., $x < y$). Future work could extend the framework with relational domains such as *Octagons* or *Polyhedra*. Because the CESK machine is parametric over the store value type, such extensions require only a new lattice definition and its domain-specific soundness proof.

6 Conclusion

We have presented a mechanically verified interval abstract interpreter in Lean 4, delivering every proposed objective and going significantly beyond the original scope.

Delivery Against Proposal.

Item	Description	Theorem	Status
D1	CESK small-step semantics	`Concrete.step`	✓proved
D2	Verified Galois connection	`galois_connection`	✓proved
D3	Soundness by induction	`global_soundness`	✓proved
S1	Widening operator	`widen_sound`	✓proved
Ext	Post-fixpoint bridge	`fixpoint_sound`	structurally complete
Ext	End-to-end corollary	`end_to_end_soundness`	structurally complete

The "structurally complete" extension theorems have no `sorry`s in their own proof bodies; their two worklist sub-lemmas are identified open problems in the voluntary extension work (see §5).

Key Contributions. The core contribution is a fully machine-checked proof that the abstract interpreter soundly simulates every concrete execution: every concrete state reachable by the CESK machine is over-approximated by the interval analysis. The proof is unconditional within the proposed scope—no axioms, no fuel parameters, no sorry.

The voluntary extension exposed a subtle gap that arises when connecting a mathematically clean soundness proof to a practical worklist algorithm: the `partial def` restriction in Lean 4 prevents unfolding the implementation in kernel proofs, motivating the `go_eq_goFuel` bridge axiom and pointing to refactoring `go` with a `termination_by` clause as the definitive fix.

The engineering discoveries along the way—canonical interval representation via `mkInterval`, weak updates as a semantic necessity rather than an optimisation, the noncomputable/computable split for α vs `step`, and the `DomPostFixpoint` weakening of the closure property for widening—demonstrate how mechanised proof actively shapes the design of verified software.